

# **Multithreading**

# Java Thread Model- Key Concepts

## 1. Java and Multithreading:

- Java is designed with multithreading as a core part of the language and runtime.
- Threads allow asynchronous execution(perform **multiple tasks concurrently** within the same process), improving efficiency by reducing idle CPU time.

## 2. Single-threaded Event Loop (Polling Model):

- In this model, a single thread repeatedly checks an event queue.
- Only one task is handled at a time, and while handling one, others must wait.
- If a task blocks (e.g., waiting for I/O), the whole program pauses.

### 3. Multithreading in Java:

- Eliminates the need for a central event loop.
- Threads operate independently—if one is blocked, others continue to run.
- Useful for I/O-bound or interactive tasks, like animations or network operations (A web browser performs multiple tasks simultaneously. Thread 1- Download webpage data, Thread 2-Load images, Thread 3- Handle user scrolling)

### 4. Thread States:

- **Running:** Actively executing.
- **Ready:** Prepared to run but waiting for CPU time.
- **Suspended:** Temporarily paused, can be resumed.
- **Blocked:** Waiting for a resource (like file or network I/O).
- **Terminated:** Has finished execution and cannot be restarted.

# Key Points on Java Thread Priorities:

## 1. Priority as a Relative Value:

- Java assigns an integer priority to each thread (typically from 1 to 10).
- These priorities **don't affect execution speed**; instead, they help determine **when a thread gets CPU time**.
- A thread's priority is only meaningful when **multiple threads compete** for CPU time.

## 2. Context Switching:

Java performs a **context switch** to determine which thread runs. This can happen in two main ways:

- **Voluntary relinquishing**: A thread pauses itself by calling methods like:

- `yield()` – suggests it's willing to pause.
- `sleep()` – pauses for a specified time.
- Waiting on I/O – e.g., reading a file or waiting for user input.
- In this case, the JVM picks the **highest-priority thread** that's ready to run.
- **Preemptive multitasking:**
  - A **higher-priority thread** preempts (interrupts) a **lower-priority thread** immediately.
  - This ensures that high-priority tasks aren't delayed.

### 3. Threads with Equal Priority:

- **On some systems (e.g., Windows):** Threads with the same priority are **time-sliced**—each gets a turn (round-robin scheduling).
- **On others (e.g., some real-time systems):** Threads must **voluntarily yield** control.
  - If a thread doesn't yield, others of the same priority might **never get CPU time**.

# Synchronization

In **multithreading**, when multiple threads access **shared resources** (like variables, or files), it's critical to **synchronize access** to avoid inconsistent or corrupted data. Java solves this using **monitors**, which are built into every object via the `synchronized` keyword.

**Synchronization** ensures that **only one thread at a time** can access a **critical section** of code (like a method that updates a shared variable). This prevents problems like **race conditions**.

- Every object in Java has an **implicit monitor**.
- When a thread calls a **synchronized method**, it **acquires the lock (monitor)** for that object.
- No other thread can call **any other synchronized method** on that same object until the lock is released.

```
class Counter {  
    int count = 0;  
    synchronized void increment() {  
        count++;           // critical section  
    }  
}
```



## Messaging in Java – Thread Communication with wait(), notify(), and notifyAll()

In Java, **thread messaging** is a way for threads to communicate and coordinate execution without using external OS-level messaging systems. Java achieves this efficiently using **wait()**, **notify()**, and **notifyAll()**—methods available to every object because they are defined in the base Object class.

- A thread **enters a synchronized method or block**, then calls **wait()**.
- It **releases the lock** on the object and enters the **waiting state**.
- Another thread, after performing its task, calls **notify()** (or **notifyAll()**) on the same object to **wake up** waiting threads.
- The waiting thread **re-acquires the lock** and resumes execution.

# The Thread Class and the Runnable Interface in Java

Java's multithreading system is centered around two main components:

1. **The Thread class** – Represents a thread of execution.
2. **The Runnable interface** – Represents a task that can be executed by a thread.

## 1. Thread Class

- A class that encapsulates the behavior of a thread.
- You can create a thread by **extending** this class and **overriding the run()** method.

## 2. Runnable Interface

- Defines a **single method: run()**.
- You implement this interface and pass it to a Thread object to execute it in a new thread.
- Preferred approach (over extending Thread) because Java does not support multiple inheritance.

# Common Thread Class Methods

| Method        | Description                                           |
|---------------|-------------------------------------------------------|
| start()       | Begins thread execution (calls run() in a new thread) |
| run()         | Entry point for the thread's task                     |
| sleep(ms)     | Pauses the thread for specified milliseconds          |
| join()        | Waits for a thread to finish                          |
| isAlive()     | Checks if a thread is still running                   |
| getName()     | Gets the thread's name                                |
| getPriority() | Gets the thread's priority                            |

```
class MyThread extends Thread {  
    public void run() {  
        System.out.println("Thread running: " + getName());  
    }  
}  
  
public class ThreadExample1 {  
    public static void main(String[] args) {  
        MyThread t1 = new MyThread();  
        t1.start(); // starts the thread  
    }  
}
```

### Example 1: Using the Thread Class (by Extending)

**Output:**  
Thread running: Thread-0

## Example 2: Using the Runnable Interface (Preferred Approach)

```
class MyRunnable implements Runnable {  
    public void run() {  
        System.out.println("Runnable thread running");  
    }  
}  
  
public class ThreadExample2 {  
    public static void main(String[] args) {  
        Thread t1 = new Thread(new MyRunnable());  
        t1.start(); // starts the thread  
    }  
}
```

Use `Runnable` when you only need to define the thread's task.

Extend `Thread` only when you need to **override other Thread methods** or when the thread **is the object**.

# Runnable

```
class MyTask implements Runnable {  
    @Override  
    public void run() {  
        // Define the task to be executed in a thread  
        System.out.println("Running the task in thread: " + Thread.currentThread().getName());  
    }  
}  
  
public class RunnableExample {  
    public static void main(String[] args) {  
        // Creating a Runnable instance  
        MyTask task = new MyTask();
```



```
// Passing the task (Runnable) to the Thread constructor
```

```
Thread thread = new Thread(task);
```

```
thread.start(); // Start the thread and execute the task
```

```
}
```

```
}
```

## Extending Thread — When You Need to Override Thread Methods or Use Thread-Specific Functionality

You would extend **Thread** when you want to:

- **Override other Thread methods** like `run()`, `start()`, `sleep()`, `join()`, or others.
- Use the **Thread class-specific functionality** (e.g., managing the thread itself, controlling thread states, or accessing thread information).
- **Treat the thread itself as an object** so you can pass it around like any other object, which may be useful in complex thread management scenarios.

When you **extend Thread**, you are directly modifying or managing the thread itself, whereas **Runnable** separates the task from the thread management.

```
class MyThread extends Thread {  
    @Override  
    public void run() {  
        // Override the run method with the specific task  
        System.out.println("Running the thread: " + getName());  
    }  
}  
  
public class ThreadExample {  
    public static void main(String[] args) {  
        // Create an instance of the thread  
        MyThread thread = new MyThread();  
        thread.start(); // Start the thread, which calls run() internally  
    }  
}
```

# The Main Thread in Java

In Java, the **main thread** is the initial thread that gets created when a program starts. It is special because it serves two main purposes:

1. **Spawning child threads:** The main thread is responsible for creating and launching other threads in your application.
2. **Shutting down the program:** It is typically the last thread to finish executing because it is often responsible for performing final tasks such as cleanup, resource release, and other shutdown operations.

While the **main thread** is created automatically by the **Java runtime** when your program starts, you can still **manipulate** it using a **Thread object**.

Java provides a method to obtain a reference to the main thread via **Thread.currentThread()**, which returns the current thread (in this case, the main thread) as an instance of Thread.

// Controlling the main Thread.

```
class CurrentThreadDemo {
```

```
    public static void main(String args[]) {
```

```
        // Obtain reference to the current (main) thread
```

```
        Thread t = Thread.currentThread();
```

```
        System.out.println("Current thread: " + t);
```

```
        // Change the name of the thread
```

```
        t.setName("My Thread");
```

```
        System.out.println("After name change: " + t);
```

```
        // Countdown with a 1-second pause between each number
```

```
try {  
    for (int n = 5; n > 0; n--) {  
        System.out.println(n);  
        Thread.sleep(1000); // Sleep for 1 second  
    }  
} catch (InterruptedException e) {  
    System.out.println("Main thread interrupted");  
}  
  
}  
  
}
```

**Output:**

Current thread:

Thread[main,5,main]

After name change: Thread[My  
Thread,5,main]

5

4

3

2

1

main → thread name

5 → priority (default = 5)

main → thread group

## Explanation of the Code:

### 1. Obtaining the Current Thread:

- `Thread t = Thread.currentThread();` obtains the current thread (the main thread in this case). This gives you a reference to the main thread, which you can manipulate like any other `Thread` object.

### 2. Changing the Thread's Name:

- `t.setName("My Thread");` changes the name of the main thread to "My Thread" from its default name, "main".

### 3. Using `sleep()`:

- Inside the loop, `Thread.sleep(1000);` pauses the main thread for 1000 milliseconds (1 second) between each countdown number. The `sleep()` method may throw an `InterruptedException`, which we handle using a try-catch block.

### 4. Handling Interruption:

- If some other thread interrupts the main thread while it is sleeping, an `InterruptedException` is thrown. In this case, we simply print a message to the console.

## Explanation of Output:

### 1. Initial Thread Information:

- The first output, `Current thread: Thread[main, 5, main]`, provides details about the main thread:
  - **Thread Name:** `main`
  - **Priority:** 5 (the default priority of threads in Java)
  - **Thread Group:** `main` (by default, threads belong to the "main" thread group)

### 2. After Changing the Name:

- After the thread name is changed using `setName()`, the output `After name change: Thread[My Thread, 5, main]` reflects the new name of the thread, which is now "My Thread".

### 3. Countdown:

- The program then counts down from 5 to 1, with a 1-second pause between each number (`Thread.sleep(1000)`).



# Creating a Thread in Java

In Java, you can create a thread in two main ways:

1. **Implementing the Runnable interface**
2. **Extending the Thread class**

## Implementing the Runnable Interface

- The **Runnable** interface is a **functional interface** that represents a unit of work to be executed concurrently. It has a single method, **run()**, which is the entry point for the thread's execution.
- To **create a thread**, you implement the **Runnable interface** and provide your **own implementation of the run() method**.
- Once you have your **Runnable object ready**, you can create a Thread object and pass the Runnable instance to it. After that, you call the start() method on the Thread object to begin execution.

## Steps to Implement Runnable:

1. **Create a class that implements Runnable.**
2. **Implement the run() method**, which contains the code that should be executed in the new thread.
3. **Create a Thread object**, passing the Runnable object to the constructor of Thread.
4. **Start the thread** by calling the start() method on the Thread object.

```
class NewThread implements Runnable { // This class defines the task (work) for the child thread

    Thread t;

    NewThread() // Constructor to create the thread    {

t = new Thread(this, "Demo Thread"); // Create a new thread, passing the current object (this) as the Runnable

        System.out.println("Child thread: " + t);

        t.start(); // Start the thread (calls run method)

    }

    public void run() {

        try {

            for (int i = 5; i > 0; i--) {

                System.out.println("Child Thread: " + i);

                Thread.sleep(500); // Pause for 500 milliseconds

            }

        }

    }

}
```

```
    }  
    catch (InterruptedException e) {  
        System.out.println("Child thread interrupted.");  
    }  
    System.out.println("Exiting child thread.");  
}  
}  
  
class ThreadDemo {  
    public static void main(String args[]) {  
        new NewThread(); // Create and start a new thread  
        try {  
            for (int i = 5; i > 0; i--) {  
                System.out.println("Main Thread: " + i);  
                Thread.sleep(1000); // Pause for 1000 milliseconds  
            }  
        }  
    }  
}
```

```
catch (InterruptedException e) {  
    System.out.println("Main thread interrupted.");  
}  
  
System.out.println("Main thread exiting.");  
}  
}
```

# Output:

Child thread: Thread[Demo Thread,5,main]

Main Thread: 5

Child Thread: 5

Child Thread: 4

Main Thread: 4

Child Thread: 3

Child Thread: 2

Main Thread: 3

Child Thread: 1

Exiting child thread.

Main Thread: 2

Main Thread: 1

Main thread exiting.

## Extending the Thread Class:

When you extend the Thread class, your new class inherits all the methods and behavior of the Thread class. You can then override the run() method to specify the code that should be executed by the new thread. You still need to call start() to begin execution of the thread.



```
class NewThread extends Thread {  
    NewThread() {  
        super("Demo Thread"); // Set the thread name using the Thread constructor  
        System.out.println("Child thread: " + this); // Print thread details  
        start(); // Start the thread  
    }  
    public void run() {  
        try {  
            for (int i = 5; i > 0; i--) {  
                System.out.println("Child Thread: " + i);  
                Thread.sleep(500); // Sleep for 500 milliseconds  
            }  
        }  
    }  
}
```

```
catch (InterruptedException e) {  
    System.out.println("Child interrupted.");  
}  
System.out.println("Exiting child thread.");  
}  
}
```

```
class ExtendThread {  
    public static void main(String args[ ]) {  
        new NewThread(); // Create and start a new thread  
        try {  
            for (int i = 5; i > 0; i--) {  
                System.out.println("Main Thread: " + i);  
                Thread.sleep(1000); // Sleep for 1 second  
            }  
        } catch (InterruptedException e) {  
            System.out.println("Main thread interrupted.");  
        }  
        System.out.println("Main thread exiting.");  
    }  
}
```

# Output:

Child thread: Thread[Demo Thread,5,main]

Main Thread: 5

Child Thread: 5

Child Thread: 4

Main Thread: 4

Child Thread: 3

Child Thread: 2

Main Thread: 3

Child Thread: 1

Exiting child thread.

Main Thread: 2

Main Thread: 1

Main thread exiting.

# Creating Multiple Threads in Java

In Java, multiple threads can be created by implementing the `Runnable` interface or extending the `Thread` class. This allows a program to perform multiple tasks concurrently, improving efficiency—especially in applications that involve waiting (like I/O operations).

In the following example, the `NewThread` class implements `Runnable`. Each instance creates a new thread with a unique name and starts it using the `start()` method.

```
class NewThread implements Runnable {  
    String name;  
    Thread t;  
    NewThread(String threadname) {  
        name = threadname;  
        t = new Thread(this, name);  
        System.out.println("New thread: " + t);  
        t.start(); // Start the thread  
    }  
}
```

```
public void run() {  
    try {  
        for (int i = 5; i > 0; i--) {  
            System.out.println(name + ": " + i);  
            Thread.sleep(1000); // Pause for 1 second  
        }  
    }  
}
```

```
catch (InterruptedException e) {  
    System.out.println(name + " interrupted.");  
}  
System.out.println(name + " exiting.");  
}  
}
```

```
public class MultiThreadDemo {  
    public static void main(String[] args) {  
        new NewThread("One");  
        new NewThread("Two");  
        new NewThread("Three");  
    }  
}
```



```
try {  
    Thread.sleep(10000); // Main thread sleeps  
} catch (InterruptedException e) {  
    System.out.println("Main thread interrupted.");  
}  
System.out.println("Main thread exiting.");  
}  
}
```

## Explanation:

- **Runnable Interface:** NewThread implements Runnable and overrides the run() method.
- **Thread Creation:** Each thread is created with a unique name and starts execution immediately.
- **Concurrent Execution:** Threads run simultaneously, sharing CPU time.
- **Thread.sleep():** Used to delay execution, showing how threads operate independently.
- **Main Thread:** Sleeps for 10 seconds to allow child threads to complete first.

New thread: Thread[One,5,main]

New thread: Thread[Two,5,main]

New thread: Thread[Three,5,main]

One: 5

Two: 5

Three: 5

One: 4

Three: 4

Two: 4

Three: 3

One: 3

Two: 3

**Output:**

Three: 2

One: 2

Two: 2

Three: 1

One: 1

Two: 1

Three exiting.

One exiting.

Two exiting.

Main thread exiting.

# isAlive( ) and join( ) method

## 1. isAlive() Method:

- Checks whether a thread is still running.
- **Syntax:** boolean isAlive()
- Returns true if the thread has **not yet terminated**, otherwise false.

## 2. join() Method:

- **Pauses the calling thread** (e.g., main) **until the thread on which join() was called finishes.**
- **Syntax:** void join() (can also accept timeout values)
- This ensures that the main thread doesn't finish before the other threads do.

```
class NewThread implements Runnable {  
    String name;  
    Thread t;  
    NewThread(String threadname) {  
        name = threadname;  
        t = new Thread(this, name);  
        System.out.println("New thread: " + t);  
        t.start();  
    }  
    public void run() {  
        try {  
            for (int i = 5; i > 0; i--) {
```

```
System.out.println(name + ": " + i);  
        Thread.sleep(1000);  
    }  
    } catch (InterruptedException e) {  
        System.out.println(name + " interrupted.");  
    }  
    System.out.println(name + " exiting.");  
    }  
}
```

```
public class DemoJoin {  
    public static void main(String args[]) {  
        NewThread ob1 = new NewThread("One");  
        NewThread ob2 = new NewThread("Two");  
        NewThread ob3 = new NewThread("Three");
```

```
System.out.println("Thread One is alive: " + ob1.t.isAlive());
System.out.println("Thread Two is alive: " + ob2.t.isAlive());
System.out.println("Thread Three is alive: " + ob3.t.isAlive());
try {
    System.out.println("Waiting for threads to finish.");
    ob1.t.join();
    ob2.t.join();
    ob3.t.join();
} catch (InterruptedException e) {
    System.out.println("Main thread Interrupted");
}
```



```
System.out.println("Thread One is alive: " + ob1.t.isAlive());  
System.out.println("Thread Two is alive: " + ob2.t.isAlive());  
System.out.println("Thread Three is alive: " + ob3.t.isAlive());  
  
System.out.println("Main thread exiting.");  
}  
  
}
```

New thread: Thread[One,5,main]

New thread: Thread[Two,5,main]

New thread: Thread[Three,5,main]

Thread One is alive: true

Thread Two is alive: true

Thread Three is alive: true

Waiting for threads to finish.

One: 5

Two: 5

Three: 5

...

Three: 1

Two: 1

**Output:**

One: 1

Three exiting.

Two exiting.

One exiting.

Thread One is alive: false

Thread Two is alive: false

Thread Three is alive: false

Main thread exiting.

The `join()` method **tells one thread to wait for another thread to finish** before continuing.

## Real-Life Analogy:

Imagine you're in a group project. You are the **main coordinator** (like the main thread), and your friends (threads) are doing different parts of the project.

You want to submit the final report **only after all friends finish their parts**. So, you wait for each friend to complete their work — that's exactly what `join()` does.

Let's say you have 3 threads: Thread One, Thread Two, and Thread Three.

**In the main method, we call:**

**`ob1.t.join();` // wait until Thread One finishes**

**`ob2.t.join();` // then wait until Thread Two finishes**

**`ob3.t.join();` // then wait until Thread Three finishes**

Until each thread finishes, the main thread **pauses and waits**. Only **after all threads are done**, the main thread resumes and prints Main thread exiting.